PROJECT REPORT · DEPARTMENT OF DATA SCIENCE AND ENGINEERING, MANIPAL UNIVERSITY JAIPUR

MedTwin

Realistic and private synthetic patient data — including the values that were never measured

A generative AI system that creates fake-but-realistic patient records from real hospital data, tested on two public medical datasets against three well-known methods, with new ways to measure whether the *missing* parts of the data are realistic and whether they leak private information.

Aarzoo Jain · Pranav Ladha

October 2026

61.5%

how often a classifier can spot our fake heart patients (50% = impossible to spot; other methods 74.9–92.8%)

0.98

usefulness of our fake data compared with real data (1.00 = just as useful)

0.17

missing-value pattern error (two real samples: 0.15; other methods 0.21–0.68)

Contents

1 Summary

2 The problem in simple words

3 Key ideas explained

4 The data

5 How MedTwin works

6 Technical specifications

7 How we test it

8 Results

9 Comparison with previous work

10 What is new in this project

11 Limitations

12 Future work

13 Project history

A How to run the project

B References

1 Summary

What. MedTwin is a program that learns from real patient records and then creates new, fake patient records that look and behave like the real ones. Nobody in the fake data is a real person. Researchers can use such data to build and test medical AI without seeing private patient files.

Why it is hard. Real hospital data has holes. Different hospitals measure different things: in our heart data, *no* Swiss patient has a cholesterol value and almost no Hungarian patient has a vessel-count result. Most existing tools fill these holes with guesses before learning, so their fake data has no holes at all — which makes it unrealistic and easy to spot.

How. MedTwin is a type of AI called a *conditional Wasserstein GAN*. For every value that can be missing it also learns a yes/no "was this measured?" flag, and it creates each patient *for a specific hospital*, so it learns each hospital's habits. We also built new tests that measure whether the holes in fake data are realistic and whether keeping them puts privacy at risk.

Results (five repeated experiments, tested on real patients the AI never saw):

- **Most realistic.** A classifier can tell our fake heart patients from real ones only 61.5% of the time (50% = pure guessing). The other three methods: 74.9%, 79.5% and 92.8%.
- **Just as useful.** A disease predictor trained only on our fake data works 97.5% as well as one trained on real data (heart disease) and 98.3% as well (diabetes), tied with the best other method.
- **Realistic holes.** The missing-value patterns in our fake data are almost as close to the real data as a second sample of real patients is. Only MedTwin puts the holes at the right hospitals.
- **No copying found.** Privacy attacks found no sign that MedTwin memorised real patients. One other method (TabDDPM) clearly did on the diabetes data.
- **An honest catch.** Keeping realistic holes makes patients with *unusual* holes slightly easier to identify (attack score 0.575 vs 0.531 without the flags). This is a new finding that we report openly.

2 The problem in simple words

Medical AI needs data that is hard to share

To build a computer program that predicts, for example, heart disease, we need many real patient records. These records are private. Laws such as HIPAA (USA) and GDPR (Europe), and hospital rules, rightly limit who can see them. So students, researchers and companies often cannot get the data they need.

The idea: synthetic data

Synthetic data means fake records produced by a computer model that has learned the patterns in real records. A good synthetic patient looks real — an older patient tends to have a lower maximum heart rate, a sick patient tends to have chest pain — but does not belong to any real person. If the fake data is good enough, people can share it instead of the real data.

A synthetic dataset must pass three tests at the same time:

1. Realistic	2. Useful	3. Private
Does it look like real data, column by column and in how columns relate?	Does a model trained on the fake data work on real patients?	Is it impossible to work out which real patients were used to make it?

The twist: real data has holes

Real medical tables are incomplete, and not at random. Whether a test was done depends on the hospital, the doctor and the patient. In the heart disease data we use:

- every Swiss patient has cholesterol recorded as 0, which really means "not measured";
- the number of major vessels (from a scan) is missing for 96–99% of patients at three of four hospitals;
- resting blood pressure is missing for 28% of patients at VA Long Beach but almost nobody elsewhere.

Most synthetic-data tools cannot handle holes, so people fill them with a typical value (for example the median) first. The fake data then has no holes, which is not how real data looks, and the filled-in values are inventions. Our project asks:

> **Project questions**
>
> 1. Can we generate realistic, useful synthetic patients *including* their holes, hospital by hospital?
> 2. How do we *measure* whether the holes in fake data are realistic?
> 3. Does keeping realistic holes create a new *privacy risk*?

3 Key ideas explained

These terms appear throughout the report.

Term	Meaning in simple words
Generative AI / generator	A model that learns from examples and then produces new examples of the same kind.
GAN (Generative Adversarial Network)	Two networks that compete: a *generator* makes fake records, a *critic* tries to tell fake from real. Both improve until the fakes are hard to spot.
Wasserstein GAN with gradient penalty (WGAN-GP)	A more stable kind of GAN. The critic gives a "realness score" instead of a yes/no answer, and a penalty keeps training smooth.
Conditioning	Telling the generator what kind of record to make. We tell it the hospital, so it learns each hospital's habits.
Missing-value flag	An extra yes/no value per column: "was this measured?" The generator learns these too, and leaves a value blank when it says "no".
Held-out patients	20% of real patients that no model ever sees during training. We test on them so the results are fair.
Seed	A number that fixes the random choices. We repeat everything with 5 seeds to check results are not luck.
TSTR	Train on Synthetic, Test on Real: train a disease predictor on fake data and test it on real held-out patients.
AUC	A score for how well a predictor separates two groups: 1.0 = perfect, 0.5 = coin flip.
Detectability	How often a classifier can tell a fake record from a real one. 50% = impossible to tell (best).
Membership inference attack (MIA)	An attack that tries to find out whether a given real patient was used to train the generator. AUC 0.5 = attack fails (good).
DCR (distance to closest record)	Checks whether fake records sit suspiciously close to training patients. 50% = no sign of copying.
Ablation	Switching off one part of the system to see how much that part matters.
Paired 95% confidence interval	A statistical test across the 5 seeds. We call a difference "clear" only when the interval does not include zero.

4 The data

We use two public medical datasets. Both are small, which is typical of medical research and makes the task harder.

	Heart disease (UCI)	Pima diabetes
Patients	920	768
Columns used	7 numbers + 7 categories + hospital	9 numbers
Hospitals	4: Cleveland, Hungary, Switzerland, VA Long Beach	1 (one clinic)
Prediction target	has heart disease (severity > 0): 55.3% yes	has diabetes: 34.9% yes
Missing values	many, depending strongly on hospital	zeros that mean "not measured"
Source	UCI Machine Learning Repository [1]	OpenML dataset 37 [2]

Heart disease: the columns

Column	Meaning	Type	Missing
`age`	Age in years	number	0.0%
`sex`	Sex	category	0.0%
`dataset`	Hospital (site)	site	0.0%
`cp`	Chest pain type	category	0.0%
`trestbps`	Resting blood pressure (mmHg)	number	6.5%
`chol`	Serum cholesterol (mg/dl)	number	22.0%
`fbs`	Fasting blood sugar > 120 mg/dl	category	9.8%
`restecg`	Resting ECG result	category	0.2%
`thalch`	Maximum heart rate (bpm)	number	6.0%
`exang`	Chest pain during exercise	category	6.0%
`oldpeak`	ST depression on ECG	number	6.7%
`slope`	Slope of ST segment	category	33.6%
`ca`	Major vessels seen on scan (0–3)	number	66.4%
`thal`	Thalassemia test result	category	52.8%
`num`	Disease severity (0 = none, 1–4)	number	0.0%

Heart disease: missing values depend on the hospital

Share of patients with the value *not measured*, per hospital (number of patients in brackets). This pattern is exactly what a realistic synthetic dataset must reproduce.

Hospital	chol	trestbps	ca	thal	slope	fbs
Cleveland (304)	0%	0%	2%	1%	0%	0%
Hungary (293)	8%	0%	99%	90%	65%	3%
Switzerland (123)	100%	2%	96%	42%	14%	61%
VA Long Beach (200)	28%	28%	99%	83%	51%	4%

Pima diabetes: the columns

768 women of Pima heritage, aged 21 or older. A value of 0 for glucose, blood pressure, skin fold, insulin or BMI is physically impossible and means “not measured”; 0 pregnancies is a real value.

Column	Meaning	Missing
`preg`	Pregnancies	0.0%
`plas`	Glucose (mg/dL)	0.7% (0 = not measured)
`pres`	Diastolic BP (mmHg)	4.6% (0 = not measured)
`skin`	Skin fold (mm)	29.6% (0 = not measured)
`insu`	Insulin (mu U/ml)	48.7% (0 = not measured)
`mass`	BMI	1.4% (0 = not measured)
`pedi`	Diabetes pedigree	0.0%
`age`	Age (years)	0.0%
`outcome`	Diabetes (1 = yes)	0.0%

5 How MedTwin works

The whole system is one pipeline. Each step is a separate, tested part of the code.

1 Load settings
A YAML file describes the dataset: columns, rules, which zeros mean missing.

2 Split
80% of patients for training, 20% held out for testing (never seen).

3 Encode
Mark missing values, add yes/no flags, scale numbers to 0–1, one-hot categories.

4 Train
Conditional WGAN-GP learns real patients, hospital by hospital (600 epochs).

5 Generate
Create 2000 fake patients, each for a hospital drawn in real proportions.

6 Decode
Back to real units; leave a value blank when its flag says "not measured".

7 Clean
Medical range check and rounding to how hospitals record values.

8 Evaluate
Realism, usefulness, privacy and missing-value tests on held-out patients.

Step by step

Settings file. Everything specific to a dataset lives in a small settings file (for example `configs/heart_disease.yaml`): which columns are numbers or categories, which column is the hospital, which zeros mean "not measured", and the medically possible range of each value. Switching to a new dataset means writing a new settings file, not changing code — that is how we added the diabetes data.

Handling missing values (the key idea). For every number column that is sometimes missing, MedTwin adds a yes/no *measured flag*. The generator learns these flags together with the values, so it learns *which* values tend to be missing and *where*. When it creates a patient and the flag says "not measured", the value is left blank. For categories, "not measured" is simply one more category.

The zero flag. 40% of heart patients have an ST depression of exactly 0. A neural network cannot easily produce one exact number, so MedTwin learns a separate yes/no "is it exactly zero?" flag.

Hospital conditioning. The hospital is not generated — it is *given* to the generator. MedTwin creates each patient for a chosen hospital, and the critic judges whether the patient is realistic *for that hospital*. This is how site-specific habits (like no cholesterol test in Switzerland) are learned.

Correlation loss. Besides fooling the critic, the generator is also penalised when the correlations between columns in its output differ from those in real data. This keeps relationships like "older → lower maximum heart rate" intact.

Cleaning. Generated values are rounded the way hospitals record them (for example ST depression to 0.1) and checked against medical ranges. The ranges were chosen to include every real measured value.

6 Technical specifications

Model architecture

Part	Specification
Generator input	random noise (128 numbers) + one-hot hospital
Generator layers	Linear 256 → LeakyReLU(0.2) → BatchNorm → Linear 512 → LeakyReLU → BatchNorm → Linear (output)
Generator outputs	Sigmoid for numbers and flags (0–1); Gumbel-softmax (τ = 0.2) per category, exactly one-hot when sampling
Critic layers	Linear 512 → LeakyReLU → Dropout 0.3 → Linear 256 → LeakyReLU → Dropout 0.3 → Linear 1 (no sigmoid)
Critic input	record + hospital, so it judges realism per hospital
Loss	Wasserstein loss + gradient penalty (λ = 10) + correlation penalty (α = 2)

Training settings

Setting	Value
Optimiser	Adam, learning rate 0.0001, betas (0.0, 0.9) for both networks
Epochs / batch size	600 epochs, batch size 64
Critic updates	5 critic updates per generator update
Records generated	2000 per run
Train / test split	80% / 20%, stratified by outcome; 5 seeds (0–4) change both the split and the model start
Training time	about 7 min (heart) and 4 min (diabetes) per run on a laptop CPU, several runs in parallel

Methods compared

Method	What it is	Training time (heart, avg.)
CTGAN [3]	Conditional tabular GAN, 300 epochs, batch 100	1.2 min
TVAE [3]	Variational autoencoder for tables, 300 epochs, batch 100	0.4 min
TabDDPM [4]	Diffusion model for tables (our compact reimplementation), 8000 steps	3.2 min
MedTwin (ours)	Conditional WGAN-GP with missing-value flags, 600 epochs	6.8 min

Software

Area	Tools and versions
Language	Python 3.9
Deep learning	PyTorch 2.8.0
Data	pandas 2.3.3, NumPy 2.0.2
Machine learning / tests	scikit-learn 1.6.1 (random forests, nearest neighbours, AUC)
Baselines	ctgan 0.12.1 (CTGAN, TVAE); TabDDPM written in this project
Statistics	SciPy (paired t-tests)
Charts	matplotlib 3.9.4, seaborn 0.13.2
Dashboard	Streamlit 1.50.0
Settings	PyYAML 6.0.3
Testing	pytest (31 automated tests)

Code structure

File / folder	Purpose
`main.py`	Runs the full pipeline once and saves the synthetic data and scores
`core/config.py`	Loads a dataset's settings file
`core/ingestion.py`	Missing-value handling, flags, scaling, encoding
`core/generator.py`	Generator, critic and WGAN-GP training
`core/medtwin.py`	Train-and-generate steps; ablation switches
`core/medical_logic.py`	Medical range check and rounding
`core/evaluation.py`	Usefulness, privacy and correlation tests on held-out patients
`core/missingness.py`	New missing-value realism scores and privacy attacks
`research/baselines.py`, `tabddpm.py`	The three comparison methods
`research/multi_seed.py`	5-seed comparison with paired statistics
`research/ablation.py`	Ablation and sensitivity studies
`research/visualizer.py`, `paper_tables.py`	Figures and paper tables
`app/dashboard.py`	Interactive Streamlit dashboard
`configs/*.yaml`	One settings file per dataset
`tests/`	31 automated tests
`paper/`	Research paper (Springer LNCS format)

Reproducibility: every experiment can be rerun from the code. When we reran all experiments after adding new measurements, all 400 previously saved results came out exactly the same.

7 How we test it

A fair test setup

- **Unseen patients.** 20% of real patients are held out and never shown to any model. Usefulness and privacy are measured against them. (Testing on patients the model saw would make results look better than they are.)
- **Five repeats.** Everything is repeated with 5 seeds. Each seed holds out different patients and starts the models differently. We report the average and the spread (± standard deviation).
- **Same treatment for all.** All four methods get the same data, the same missing-value flags, the same cleaning and the same tests.
- **Paired statistics.** Methods are compared seed by seed. A difference is called *clear* only if its 95% confidence interval does not include zero.

What we measure

Measure	Question it answers	Best value	Group
Fidelity	Do single columns look real? (overlap of histograms)	higher, max 100%	realism
Detectability	Can a classifier tell fake from real?	50%	realism
Correlation gap	Do relationships between columns match?	0	realism
TSTR AUC	How good is a disease predictor trained on fake data, tested on real patients?	higher	usefulness
AUC ratio	TSTR AUC divided by the same predictor trained on real data	1.00	usefulness
DCR	Are fake records closer to training patients than to unseen ones?	50%	privacy
MIA AUC	Can an attacker tell which patients were used for training?	0.5	privacy
Missing-rate gap new	Is each column missing as often as in real data?	0	missing values
Site gap new	Is each column missing as often *at each hospital*?	0	missing values
Pattern distance new	Do whole patterns of holes (which values are missing together) match?	0	missing values
Rare-pattern MIA new	Are patients with unusual holes easier to identify?	0.5	privacy of holes
Unique-pattern leak new	Do one-of-a-kind hole patterns of training patients reappear?	0	privacy of holes

> **Detectability is not privacy**
>
> Some papers call the "can a classifier spot the fakes?" score a privacy score. It is not: a model that simply copied real patients would be impossible to spot and still leak everything. We use it only for realism, and judge privacy with DCR and membership attacks.

The new missing-value measures, in simple words

A patient's **missingness pattern** is the list of which of their values are blank, plus their hospital — for example "Hungary: cholesterol measured, vessel scan missing, thalassemia missing".

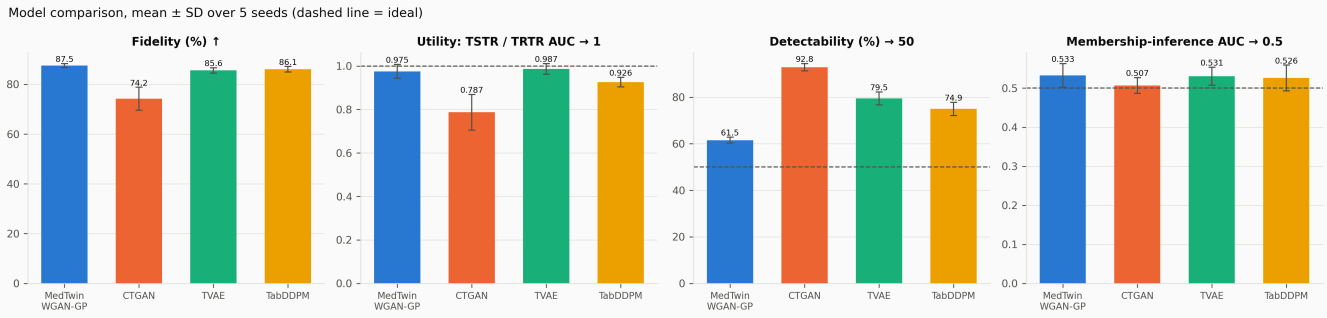
- **Pattern distance** compares how often each pattern appears in fake and in real data (0 = identical, 1 = completely different). Because the score depends on sample size, we also compute it between two groups of *real* patients. A fake dataset that scores close to this real-vs-real value is as good as more real data.
- **Rare-pattern attack** runs the membership attack only on patients whose pattern is rare (held by at most 2% of patients) and compares with everyone else. If keeping holes hurts privacy, rare patients suffer most.

8 Results

All numbers are averages over 5 seeds ± standard deviation, measured on real patients the models never saw. Arrows show what is better: ↑ higher, ↓ lower, → closer to the given value. Best values in bold are described in the text.

8.1 Heart disease: MedTwin against the other methods

Method	Fidelity % ↑	Detect. % →50	TSTR AUC ↑	AUC ratio →1	DCR % →50	MIA AUC →0.5
MedTwin (ours)	87.5 ± 0.8	61.5 ± 1.3	0.863 ± 0.023	0.975 ± 0.032	52.9 ± 2.8	0.533 ± 0.030
TVAE	85.6 ± 1.1	79.5 ± 2.8	0.873 ± 0.022	0.987 ± 0.024	50.1 ± 3.4	0.531 ± 0.023
TabDDPM	86.1 ± 1.1	74.9 ± 2.8	0.820 ± 0.030	0.926 ± 0.022	52.7 ± 3.0	0.526 ± 0.033
CTGAN	74.2 ± 4.6	92.8 ± 1.6	0.696 ± 0.069	0.787 ± 0.082	51.1 ± 3.4	0.507 ± 0.020



Heart disease comparison, averages over 5 seeds with spread. Dashed lines show the ideal value.

- **Realism:** MedTwin has the best fidelity (87.5%) and by far the lowest detectability (61.5%, against 74.9–92.8%). Both differences are clear against every method.
- **Usefulness:** AUC ratio 0.975 — statistically tied with TVAE (0.987), clearly better than CTGAN (0.787).
- **Privacy:** DCR and MIA are close to the ideal for all methods; no method copied a real patient.

8.2 Diabetes: the same picture

Method	Fidelity % ↑	Detect. % →50	TSTR AUC ↑	AUC ratio →1	DCR % →50	MIA AUC →0.5
MedTwin (ours)	90.9 ± 0.8	64.0 ± 2.6	0.801 ± 0.030	0.983 ± 0.041	51.8 ± 1.8	0.494 ± 0.021
TVAE	88.0 ± 1.7	70.0 ± 1.4	0.806 ± 0.013	0.989 ± 0.028	54.0 ± 2.4	0.518 ± 0.024
TabDDPM	85.4 ± 1.3	55.9 ± 2.6	0.781 ± 0.050	0.957 ± 0.026	60.8 ± 4.9	0.803 ± 0.028
CTGAN	77.8 ± 3.3	86.3 ± 2.7	0.568 ± 0.062	0.697 ± 0.075	49.7 ± 7.7	0.505 ± 0.028

- MedTwin again has the best fidelity (90.9%) and ties for usefulness (0.983).
- TabDDPM is the hardest to spot (55.9%) **but it memorised its training patients**: 60.8% of its records sit closer to training patients and the membership attack reaches AUC 0.803. MedTwin shows no such leak (0.494). This also shows why detectability alone must not be read as privacy.

8.3 Are the missing values realistic?

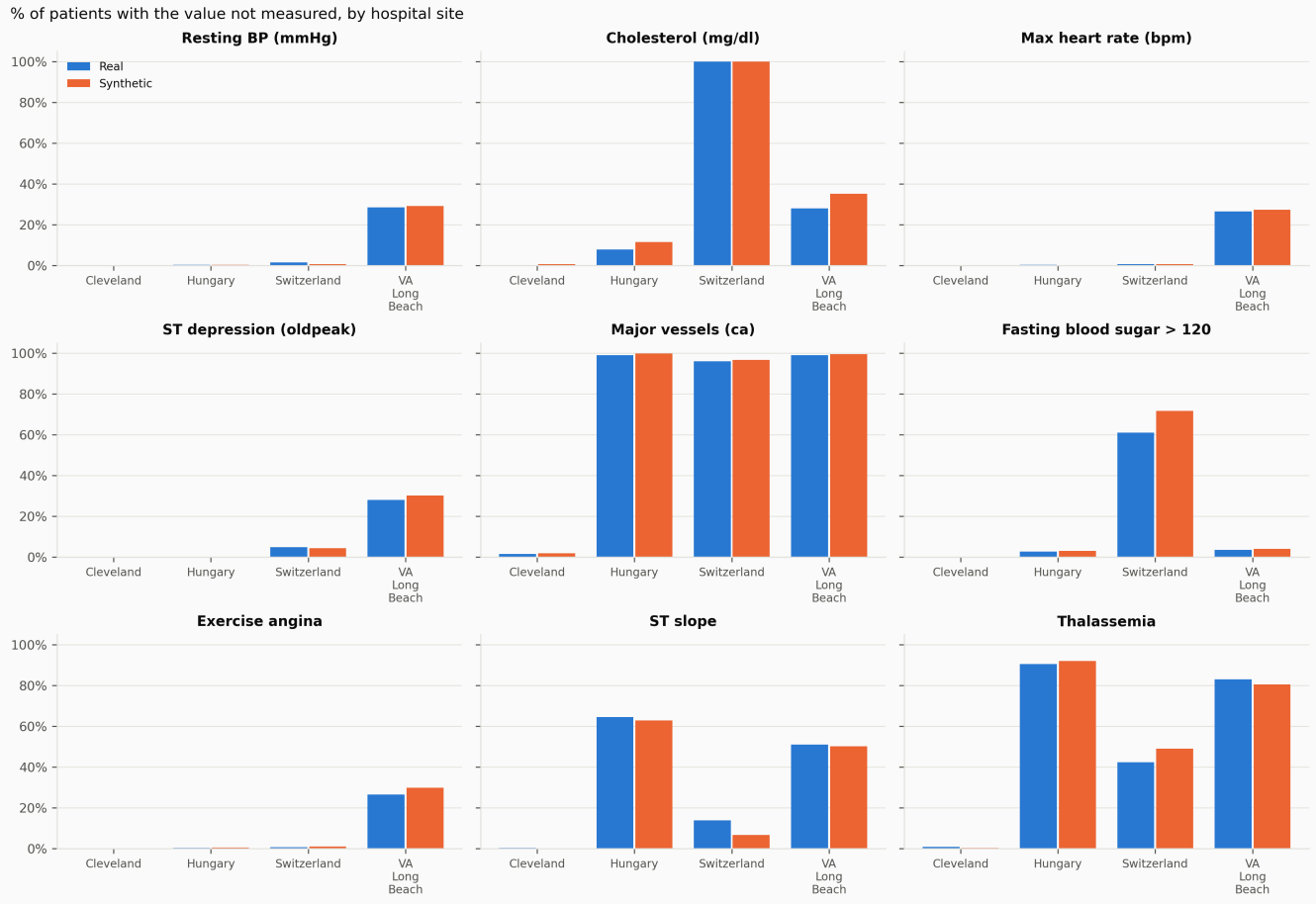
Heart disease

	Rate gap ↓	Site gap ↓	Pattern dist. ↓	MIA rare →0.5	MIA common →0.5
Two real samples (reference)	0.020	0.024	0.152		
MedTwin (ours)	0.021 ± 0.002	0.022 ± 0.001	0.174 ± 0.006	0.559 ± 0.037	0.545 ± 0.037
TVAE	0.028 ± 0.005	0.035 ± 0.002	0.209 ± 0.009	0.566 ± 0.060	0.541 ± 0.031
TabDDPM	0.032 ± 0.005	0.101 ± 0.004	0.487 ± 0.028	0.501 ± 0.032	0.539 ± 0.037
CTGAN	0.059 ± 0.005	0.131 ± 0.006	0.683 ± 0.014	0.544 ± 0.070	0.515 ± 0.029

Diabetes (one clinic, so there is no site gap)

	Rate gap ↓	Site gap ↓	Pattern dist. ↓	MIA rare →0.5	MIA common →0.5
Two real samples (reference)	0.030	–	0.085		
MedTwin (ours)	0.019 ± 0.002	–	0.063 ± 0.006	0.501 ± 0.177	0.506 ± 0.017
TVAE	0.031 ± 0.005	–	0.118 ± 0.022	0.549 ± 0.152	0.518 ± 0.029
TabDDPM	0.029 ± 0.010	–	0.111 ± 0.013	0.682 ± 0.081	0.799 ± 0.028
CTGAN	0.089 ± 0.033	–	0.299 ± 0.032	0.501 ± 0.202	0.509 ± 0.023

- MedTwin's pattern distance (0.174) is close to the gap between two real samples (0.152) and clearly better than every other method in all 5 seeds (TVAE 0.209, TabDDPM 0.487, CTGAN 0.683).
- Only MedTwin gets the hospital right: its site gap (0.022) matches real data (0.024). The other methods were given the same flags, but they do not know which hospital a hole belongs to, so they put holes in the wrong places.
- Without the flags, the pattern distance jumps to 0.693; without hospital conditioning it rises to 0.223 and the site gap to 0.028.



Share of heart patients with each value not measured, per hospital: real (blue) and MedTwin (orange).

8.4 Does keeping missing values cost privacy?

Yes, a little, for one group. On heart disease about 64 of the 368 attacked patients have a *rare* missingness pattern. For them, the membership attack reaches AUC **0.575** against MedTwin, compared with 0.531 when MedTwin is trained without the missing-value flags — higher in all 5 seeds. For patients with common patterns (0.542 vs 0.516) and overall (0.532 vs 0.515) there is no clear difference. One-of-a-kind patterns of training patients do not reappear more often than those of unseen patients, so the model is not copying patterns. On diabetes only about 8 patients per test have rare patterns, too few for a conclusion.

> **What this means**
>
> Realistic holes make the data more realistic and keep overall privacy unchanged, but patients with *unusual* holes become slightly easier to single out. Data owners should know this; merging rare patterns before release is a possible fix to test next.

8.5 What each part contributes (ablation)

Heart disease (* = clear difference from the full model)

Variant	Detect. % →50	Pattern dist. ↓	Corr. gap ↓	AUC ratio →1	MIA AUC →0.5
Full MedTwin	62.2 ± 1.7	0.171 ± 0.006	0.041 ± 0.007	0.974 ± 0.028	0.532 ± 0.030
without missing-value flags	84.2 ± 1.2*	0.693 ± 0.009*	0.071 ± 0.007*	0.989 ± 0.025	0.515 ± 0.027
without zero flag	74.9 ± 0.9*	–	–	0.978 ± 0.032	0.539 ± 0.038
without hospital conditioning	65.6 ± 1.7*	0.223 ± 0.012*	0.046 ± 0.005	0.975 ± 0.038	0.527 ± 0.025
without correlation loss	65.2 ± 2.4	–	0.075 ± 0.018*	0.982 ± 0.011	0.525 ± 0.032

Diabetes

Variant	Detect. % →50	Pattern dist. ↓	Corr. gap ↓	AUC ratio →1	MIA AUC →0.5
Full MedTwin	64.0 ± 2.6	0.063 ± 0.006	0.034 ± 0.007	0.983 ± 0.041	0.494 ± 0.021
without missing-value flags	81.5 ± 1.6*	0.486 ± 0.018*	0.042 ± 0.004*	0.993 ± 0.032	0.524 ± 0.055
without correlation loss	63.8 ± 2.9	–	0.063 ± 0.008*	0.964 ± 0.032	0.508 ± 0.029

- **Missing-value flags** matter most: without them detectability rises from 62.2% to 84.2% (heart) and from 64.0% to 81.5% (diabetes). Filling holes with typical values makes fake patients easy to spot.
- **Zero flag** and **hospital conditioning** also clearly help realism (74.9% and 65.6% without them).
- **Correlation loss** does not change realism or usefulness, but without it the correlation gap nearly doubles (0.041 → 0.075 heart; 0.034 → 0.063 diabetes).
- No part changes usefulness or overall privacy. The medical range check never removed a patient: it is only a safety net.

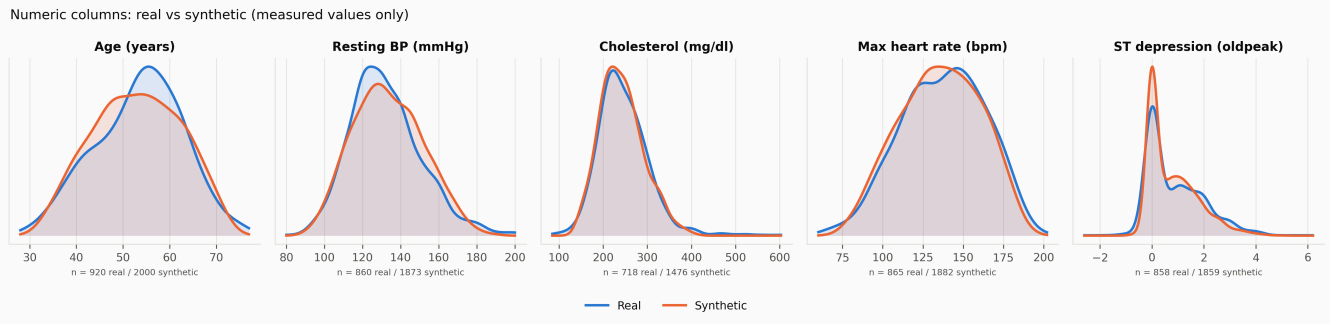
8.6 Do results depend on our chosen settings?

Setting (heart disease)	Fidelity %	Detect. %	AUC ratio	DCR %	MIA AUC
Default (weight 2, 600 epochs)	87.2 ± 0.7	62.2 ± 1.7	0.974 ± 0.028	53.2 ± 2.0	0.532 ± 0.030
Correlation weight 0.5	87.0 ± 0.8	62.8 ± 2.8	0.983 ± 0.021	53.3 ± 2.1	0.531 ± 0.023
Correlation weight 8	87.9 ± 0.5	62.8 ± 2.3	0.989 ± 0.031	53.9 ± 2.6	0.525 ± 0.032
300 epochs	87.0 ± 0.5	63.6 ± 2.8	0.961 ± 0.034	52.2 ± 2.5	0.524 ± 0.035
1200 epochs	87.2 ± 0.5	63.0 ± 2.2	0.984 ± 0.017	55.3 ± 2.6*	0.548 ± 0.024*

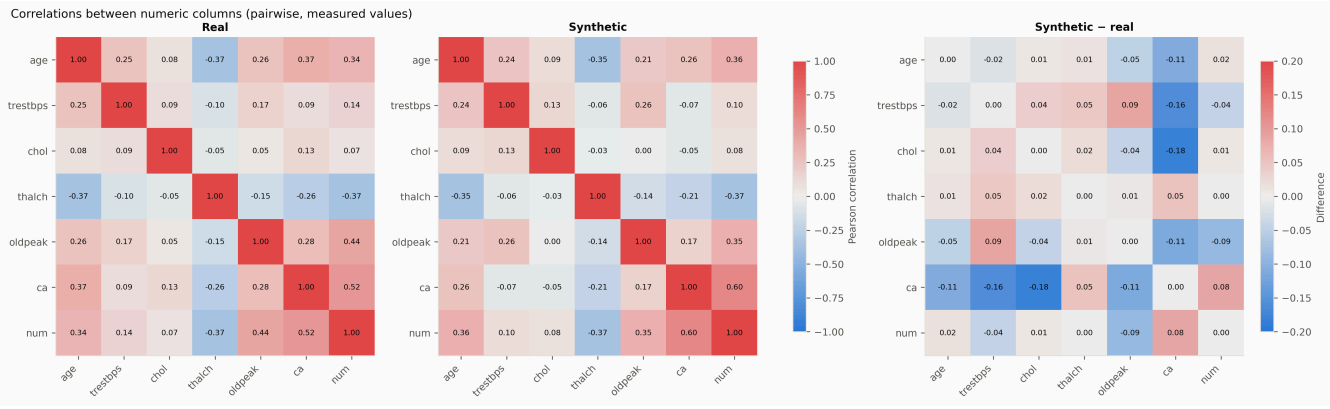
Changing the correlation weight (0.5 or 8 instead of 2) or training for 300 epochs makes no clear difference. Training twice as long (1200 epochs) starts to memorise: DCR and MIA rise clearly. So 600 epochs is a good

choice, and the results do not depend on lucky settings.

8.7 A closer look at the generated values



Distributions of the number columns (heart disease): real (blue) vs MedTwin (orange), measured values only.



Correlations between number columns: real, MedTwin, and the difference (heart disease).

9 Comparison with previous work

We searched for work from the last five years on the same datasets or the same problem. Exact number-for-number comparison is usually not possible, because each paper uses different data, splits and measures. The table shows what each did and how our project relates.

Work	What they did	Their result	How we compare
Wan Abu Bakar et al., 2025 (IJACSA) [5]	CTGAN on Cleveland heart data (303 patients); missing values filled with most common value; single run; GAN saw all patients	Accuracy train-on-fake, test-on-real: 56% (5-class)	We use all 4 hospitals, unseen test patients and 5 seeds. Our CTGAN baseline is also weak; MedTwin clearly beats it.
Subedi et al., 2025 (IEEE ICUIS) [6]	CTGAN on Pima diabetes	Random forest accuracy 82% (real) → 63% (fake), i.e. 77% kept	MedTwin keeps 98% of real-data performance (AUC ratio, 5 seeds). Setups differ, so this is indicative.
Alqulaity & Yang, 2024 (Sensors) [7]	Enhanced conditional GAN, 218 cardiovascular patients, one hospital	KS statistic 0.390 (TVAE 0.386); no usefulness or privacy test	Our evaluation covers realism, usefulness and privacy with unseen patients and repeats.
Lazzaro et al., 2026 (J. Comp. Science) [8]	Bayesian network on 3 UCI heart sites, trained on Cleveland only	Augmentation results	We model all 4 hospitals including the very incomplete ones.
HealthGen, 2022 [9]; MissDiff, 2023 [10]; Wang et al., 2025 (IEEE TKDE) [11]; IGAMT, 2024 (AAAI) [12]	Generators that keep missing values (time series, diffusion, plug-in, private EHR)	Show that keeping missingness matters	Same goal. Our additions: hospital-conditioned holes, standard scores for hole realism, and the first test of the privacy cost of holes. We have not yet compared against them directly.

In short: on the same datasets, our results and testing are stronger than recent published work. On the core idea (keeping missing values), other groups were first, so our novelty is in *measuring* realistic holes, in hospital conditioning, and in showing their privacy cost.

10 What is new in this project

1. **Scores for realistic missing values.** Missing rate per column, per hospital, and the full pattern of which values go missing together — each compared with the gap between two real samples, so "good" has a clear meaning. Existing papers do not use a common measure for this.
2. **A privacy audit for missing values.** The first test of whether realistic holes make patients easier to identify. We found a small, consistent cost for patients with rare patterns that standard privacy tests do not show.
3. **Hospital-aware generation of holes.** MedTwin is the only tested method that puts holes at the right hospital, because each patient is generated for a given hospital.
4. **A careful, honest evaluation.** Unseen test patients, five repeats, paired statistics, ablation of every part, a settings check, and a clear separation of realism from privacy.

11 Limitations

- **Small datasets.** Both datasets have fewer than 1000 patients and one table each. Larger hospital databases and data over time may behave differently.
- **Still detectable.** A classifier can still spot our fake heart patients 62% of the time (50% would be perfect).
- **Privacy is tested, not guaranteed.** We use attacks, not a mathematical guarantee such as differential privacy. Passing these attacks does not prove the data is safe.
- **Missing-value privacy audit needs enough rare patients.** It works on heart disease but not on diabetes.
- **Not compared with every method.** The newest missing-value generators (MissDiff, Wang et al.) and TabSyn were not run in our benchmark. TabDDPM is our own compact version.
- **One reference needs checking.** The original heart disease paper (Detrano et al., 1989) is marked to verify.

12 Future work

1. Compare directly with MissDiff and Wang et al., the closest previous work.
2. Test fixes for the privacy cost, such as merging rare missing-value patterns before release.
3. Add formal privacy (differentially private training) and measure what it costs in realism.
4. Generate patients for a hospital the model has never seen, to support data sharing between hospitals.
5. Move to larger and time-based hospital data.

13 Project history

The project grew in steps. An early version (April 2026) used 9 columns, filled missing values, and tested on patients the model had seen. It was rebuilt to handle all 15 columns with missing values and hospitals, to test on unseen patients, and to compare against other methods over five seeds. The recorded steps since then:

Date	Step
2026-09-29	Snapshot before 15-column, missing-data and conditional-generation work
2026-09-29	Generate all 15 columns with 'not measured' handling and site conditioning
2026-09-29	Oldpeak zero flag, 15-column 5-seed comparison, paper figures
2026-09-29	Experiment: move dataset-specific settings into configs/heart_disease.yaml
2026-09-29	Experiment: run the pipeline on Pima diabetes from configs/diabetes.yaml
2026-10-15	Add ablation switches and a multi-seed ablation runner
2026-10-15	Rename privacy audit to detectability, add sensitivity study, draft related work
2026-10-15	Add ablation, sensitivity and diabetes 5-seed results; fix mixed-name loading
2026-10-15	Add correlation-gap metric, rerun correlation ablation, write LNCS paper
2026-10-15	Add missing-value fidelity scores and privacy audit; refocus paper on them

A How to run the project

From the project folder (Windows):

Task	Command
Install	`python -m venv venv && venv\Scripts\pip install -r requirements.txt`
Run the pipeline (heart)	`python main.py`
Run on diabetes	`python main.py --config configs/diabetes.yaml`
Dashboard	`streamlit run app/dashboard.py`
5-seed comparison	`python research/multi_seed.py`
Ablations	`python research/ablation.py`
Settings check	`python research/ablation.py --study sensitivity`
Figures	`python research/visualizer.py --benchmark`
Paper tables	`python research/paper_tables.py`
Tests	`python -m pytest tests -q`
This report	`python docs/build_report.py`

A full 5-seed comparison takes about an hour on a laptop when seeds run in parallel. Results are saved per run, so an interrupted run continues where it stopped.

B References

1. Detrano R. et al. International application of a new probability algorithm for the diagnosis of coronary artery disease. Am. J. Cardiology 64(5), 1989 (to verify). UCI Heart Disease dataset.
2. Smith J.W. et al. Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. Proc. SCAMC, 1988. OpenML dataset 37.
3. Xu L. et al. Modeling tabular data using conditional GAN (CTGAN, TVAE). NeurIPS 2019.
4. Kotelnikov A. et al. TabDDPM: Modelling tabular data with diffusion models. ICML 2023.
5. Wan Abu Bakar W.A. et al. Evaluating CTGAN-generated synthetic data for heart disease prediction. IJACSA 16(12), 2025.
6. Subedi P. et al. Synthetic data for health risk prediction: an evaluation using machine learning. IEEE ICUIS 2025.
7. Alqulaity M., Yang P. Enhanced conditional GAN for synthetic tabular data in mobile-based cardiovascular healthcare. Sensors 24(23), 2024.
8. Lazzaro et al. Assessing the impact of synthetic data generated by Bayesian networks on heart disease prediction. J. Computational Science 99, 2026.
9. Bing S. et al. Conditional generation of medical time series for extrapolation to underrepresented populations (HealthGen). arXiv 2201.08186, 2022.
10. Ouyang Y. et al. MissDiff: Training diffusion models on tabular data with missing values. ICML 2023 workshop.
11. Wang X., Asif H., Gupta S., Vaidya J. Data synthesis reinvented: preserving missing patterns for enhanced analysis. IEEE TKDE 37(7), 2025.
12. Wang W. et al. IGAMT: Privacy-preserving EHR synthesization with heterogeneity and irregularity. AAAI 2024.
13. Gulrajani I. et al. Improved training of Wasserstein GANs. NeurIPS 2017.
14. Stadler T., Oprisanu B., Troncoso C. Synthetic data – anonymisation groundhog day. USENIX Security 2022.
15. Esteban C., Hyland S., Rätsch G. Real-valued (medical) time series generation with recurrent conditional GANs (TSTR). arXiv 1706.02633, 2017.